

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Специальность «Программная инженерия»

Кафедра инженерной психологии и эргономики

Учебная дисциплина «Метрология, стандартизация и сертификация (в
информационных технологиях)»

Отчет

по лабораторной работе №1
«Метрики размера программ»

Подготовил:

Студент гр. 410902
Большакова М.В.
Барбашёва А.А.

Проверил:

Болтак С. В.

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Изучить метрики размера программ на примере метрики Холстеда

2 ЗАДАНИЕ

Написать программу-парсер. Парсер может быть реализован на любом языке программирования и должен иметь графический интерфейс. Для анализа взять код на заданном языке. На основе анализа программа должна рассчитать метрики Холстеда: 6 базовых и 3 расширенных. Результат работы программы - таблица, как в примере, в которой должны отображаться 6 основных метрик Холстеда. Отдельно выводятся 3 расширенные метрики.

В данной работе представлен расчет метрик Холстеда для программы, демонстрирующей работу операторов языка C++.

Код анализируемой программы, написанный на языке C++:

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
from math import log2

# импортируем наш парсер из соседнего файла
from metrik1 import parsecpp

def calculate_metrics(operators, operands):
    # считаем уникальные элементы (словари)
    n1 = len(operators)
    n2 = len(operands)

    # считаем сколько раз они всего встречаются
    N1 = sum(operators.values())
    N2 = sum(operands.values())

    # базовые метрики
    n = n1 + n2
    N = N1 + N2

    # объем (защита от логарифма нуля, если код пустой)
    V = N * log2(n) if n > 0 else 0

    # расширенные метрики по формулам из метода
    # если операндов нет, ставим 0, чтобы не словить ошибку деления на ноль
    D = (n1 / 2) * (N2 / n2) if n2 > 0 else 0
    E = D * V
    T = E / 18 # 18 - это число Струда для времени

    # возвращаем словарик, сразу всё округляем до целых (int), чтобы без точек
    return {
        "n1": int(n1), "n2": int(n2),
        "N1": int(N1), "N2": int(N2),
```

```

    "n": int(n), "N": int(N), "V": int(V),
    "D": int(D), "E": int(E), "T": int(T)
}

```

```

class HalsteadApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Анализатор метрик Холстеда")
        self.root.geometry("1150x850")
        self.root.minsize(950, 750)

        # тут будем хранить наши посчитанные данные
        self.operators = {}
        self.operands = {}
        self.metrics = {}

        self.setup_ui()

    def setup_ui(self):
        # верхняя менюшка с кнопками
        top_panel = ttk.Frame(self.root, padding=10)
        top_panel.pack(fill="x")

        ttk.Button(top_panel, text="Выбрать файл", command=self.load_file).pack(side="left", padx=5)
        ttk.Button(top_panel, text="Рассчитать", command=self.run_analysis).pack(side="left", padx=5)
        ttk.Button(top_panel, text="Очистить", command=self.reset_all).pack(side="left", padx=5)

        self.filepath_lbl = ttk.Label(top_panel, text="Файл не выбран", foreground="gray")
        self.filepath_lbl.pack(side="left", padx=15)

        # окошко для кода
        code_frame = ttk.LabelFrame(self.root, text="Исходный код (C++)", padding=8)
        code_frame.pack(fill="both", expand=False, padx=10, pady=(0, 10))

        self.code_text = tk.Text(code_frame, wrap="none", font=("Menlo", 12), height=10)
        self.code_text.pack(side="left", fill="both", expand=True)

        # прикручиваем скролл к текстовому полю
        scroll_y = ttk.Scrollbar(code_frame, orient="vertical", command=self.code_text.yview)
        scroll_y.pack(side="right", fill="y")
        self.code_text.configure(yscrollcommand=scroll_y.set)

        # блок под две таблицы
        tables_frame = ttk.Frame(self.root)
        tables_frame.pack(fill="both", expand=True, padx=10, pady=(0, 10))

        # 1. таблица операторов (слева)
        op_frame = ttk.LabelFrame(tables_frame, text="Операторы", padding=5)
        op_frame.pack(side="left", fill="both", expand=True, padx=(0, 5))

        self.tree_op = ttk.Treeview(op_frame, columns=("j", "name", "count"), show="headings")
        self.tree_op.heading("j", text="j")
        self.tree_op.heading("name", text="Оператор")
        self.tree_op.heading("count", text="flj")

        self.tree_op.column("j", width=50, anchor="center")

```

```

self.tree_op.column("name", width=250, anchor="w")
self.tree_op.column("count", width=80, anchor="center")

self.tree_op.pack(side="left", fill="both", expand=True)

op_scroll = ttk.Scrollbar(op_frame, orient="vertical", command=self.tree_op.yview)
op_scroll.pack(side="right", fill="y")
self.tree_op.configure(yscrollcommand=op_scroll.set)

# 2. таблица операндов (справа)
opd_frame = ttk.LabelFrame(tables_frame, text="Операнды", padding=5)
opd_frame.pack(side="right", fill="both", expand=True, padx=(5, 0))

self.tree_opd = ttk.Treeview(opd_frame, columns=("i", "name", "count"), show="headings")
self.tree_opd.heading("i", text="i")
self.tree_opd.heading("name", text="Операнд")
self.tree_opd.heading("count", text="f2i")

self.tree_opd.column("i", width=50, anchor="center")
self.tree_opd.column("name", width=250, anchor="w")
self.tree_opd.column("count", width=80, anchor="center")

self.tree_opd.pack(side="left", fill="both", expand=True)

opd_scroll = ttk.Scrollbar(opd_frame, orient="vertical", command=self.tree_opd.yview)
opd_scroll.pack(side="right", fill="y")
self.tree_opd.configure(yscrollcommand=opd_scroll.set)

# вывод итоговых метрик снизу
metrics_frame = ttk.Frame(self.root, padding=10)
metrics_frame.pack(fill="x", padx=10, pady=(0, 10))

self.lbl_basic = ttk.Label(
    metrics_frame,
    text="Словарь программы η = ... \n Длина программы N = ... \n Объем программы V = ...",
    justify="left", font=("Helvetica Neue", 13)
)
self.lbl_basic.pack(side="left", fill="x", expand=True)

self.lbl_extended = ttk.Label(
    metrics_frame,
    text="Расширенные метрики: \n Сложность D = ... \n Трудоемкость E = ... \n Время T = ...",
    justify="left", font=("Helvetica Neue", 12)
)
self.lbl_extended.pack(side="left", fill="x", expand=True)

def load_file(self):
    # диалог выбора файла
    filepath = filedialog.askopenfilename(
        title="Выбери код для анализа",
        filetypes=[("C++ файлы", "*.cpp *.h *.cc"), ("Все файлы", "*.*")]
    )
    if not filepath:
        return

    try:
        # читаем файл и закидываем в текстовое поле

```

```

with open(filepath, "r", encoding="utf-8") as file:
    content = file.read()
self.code_text.delete("1.0", tk.END)
self.code_text.insert("1.0", content)
self.filepath_lbl.config(text=filepath, foreground="")
except Exception as e:
    messagebox.showerror("ошибочка", f"Не получилось открыть файл:\n{e}")

def run_analysis(self):
    # забираем текст из поля ввода
    code = self.code_text.get("1.0", tk.END).strip()
    if not code:
        messagebox.showwarning("Пусто", "Сначала загрузи или вставь код!")
        return

    try:
        # парсим и считаем
        self.operators, self.operands = parsecpp(code)
        self.metrics = calculate_metrics(self.operators, self.operands)

        # обновляем UI
        self.fill_tables()
        self.update_labels()
    except Exception as e:
        messagebox.showerror("Ошибка парсинга", f"Что-то пошло не так при анализе:\n{e}")

def fill_tables(self):
    # чистим старые записи
    for row in self.tree_op.get_children():
        self.tree_op.delete(row)
    for row in self.tree_opd.get_children():
        self.tree_opd.delete(row)

    # сортируем словари по убыванию значений (чтобы частые были сверху)
    sorted_ops = sorted(self.operators.items(), key=lambda item: item[1], reverse=True)
    sorted_opds = sorted(self.operands.items(), key=lambda item: item[1], reverse=True)

    # заполняем операторы
    for j, (op, count) in enumerate(sorted_ops, start=1):
        self.tree_op.insert("", "end", values=(f"{j}.", op, count))

    # заполняем операнды
    for i, (opd, count) in enumerate(sorted_opds, start=1):
        self.tree_opd.insert("", "end", values=(f"{i}.", opd, count))

    # добавляем итоговую строчку с суммами в самый низ
    m = self.metrics
    self.tree_op.insert("", "end", values=(f"η1 = {m['n1']}", "", f"N1 = {m['N1']}"))
    self.tree_opd.insert("", "end", values=(f"η2 = {m['n2']}", "", f"N2 = {m['N2']}"))

def update_labels(self):
    m = self.metrics

    # обновляем текст снизу
    self.lbl_basic.config(
        text=(
            f"Словарь программы η = {m['n1']} + {m['n2']} = {m['n']}\n"

```

```

        f"Длина программы N = {m['N1']} + {m['N2']} = {m['N']}\n"
        f"Объем программы V = {m['N']} * log2({m['n']}) = {m['V']}"
    )
)

self.lbl_extended.config(
    text=(
        "Расширенные метрики:\n"
        f"Сложность D = {m['D']}\n"
        f"Трудоемкость E = {m['E']}\n"
        f"Время T = {m['T']} сек"
    )
)

def reset_all(self):
    # сбрасываем вообще всё
    self.code_text.delete("1.0", tk.END)
    self.filepath_lbl.config(text="Файл не выбран", foreground="gray")

    for row in self.tree_op.get_children():
        self.tree_op.delete(row)
    for row in self.tree_opd.get_children():
        self.tree_opd.delete(row)

    self.lbl_basic.config(text="Словарь программы η = ... \nДлина программы N = ... \nОбъем
программы V = ...")
    self.lbl_extended.config(text="Расширенные метрики:\nСложность D = ... \nТрудоемкость E =
... \nВремя T = ...")

    self.operators.clear()
    self.operands.clear()
    self.metrics.clear()

if __name__ == "__main__":
    window = tk.Tk()
    app = HalsteadApp(window)
    window.mainloop()

```

3 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

Выбрать файл

Рассчитать

Очистить

/Users/margo/University 3nd Grad/Метра/LR1/Holsted-master/syntaxC++.cpp

Исходный код (C++)

```
// Демонстрационная программа: разнообразные синтаксические конструкции C++
#include <iostream>
#include <string>
using namespace std;

enum Color { RED, GREEN, BLUE }; // перечисление

struct Point { // структура
    int x;
    int y;
};
```

Операторы

Операнды

j	Оператор	f1j
1.	;	80
2.	<<	68
3.	{}	27
4.	=	22
5.	()	21
6.	,	14
7.	return	12
8.	==	8
9.	*	8
10.	:	8
11.	if	6
12.	++	6
13.	for	5
14.	.	5
15.	square()	4
16.	<=	4
17.	factorial()	3
18.	power()	3
19.	case	3
20.	+=	3
21.	%	3
22.	<	3
23.	>	3
24.	&	3

Словарь программы η = 48 + 86 = 134
Длина программы N = 353 + 261 = 614
Объем программы V = 614 * log2(134) = 4338

i	Операнд	f2i
1.	cout	25
2.	endl	22
3.	i	17
4.	0	14
5.	a	11
6.	1	8
7.	3	8
8.	n	7
9.	j	7
10.	x	6
11.	y	6
12.	5	5
13.	value	5
14.	2	4
15.	counter	4
16.	attempts	4
17.	p1	4
18.	Color	3
19.	GREEN	3
20.	Point	3
21.	result	3
22.	p	3
23.	fact	3
24.	sum	3

Расширенные метрики:
Сложность D = 72
Трудоемкость E = 316009
Время T = 17556 сек

Рисунок 1 — Окно программы

Операторы

Операнды

j	Оператор	f1j
26.	movePoint()	2
27.	isPositive()	2
28.	colorName()	2
29.	"=	2
30.	?	2
31.	while	1
32.	switch	1
33.	catch	1
34.	default	1
35.	main()	1
36.	continue	1
37.	break	1
38.	try	1
39.	throw	1
40.	runtime_error()	1
41.	what()	1
42.	goto	1
43.	&&	1
44.	--	1
45.	-	1
46.	/	1
47.	[]	1
48.	do...while	1

η1 = 48
N1 = 353

i	Операнд	f2i
64.	"Значение через указатель: "	1
65.	"Положительное? "	1
66.	"да"	1
67.	"нет"	1
68.	"2^5 = "	1
69.	"3^2 (по умолчанию) = "	1
70.	"square(2.5) = "	1
71.	"Деление на нули!"	1
72.	"Ошибка: "	1
73.	"goto итерация: "	1
74.	20	1
75.	4	1
76.	6	1
77.	8	1
78.	7	1
79.	42	1
80.	2.5	1
81.	100	1
82.	std	1
83.	nullptr	1
84.	false	1
85.	string	1
86.	runtime_error	1

η2 = 86
N2 = 261

Словарь программы η = 48 + 86 = 134
Длина программы N = 353 + 261 = 614
Объем программы V = 614 * log2(134) = 4338

Расширенные метрики:
Сложность D = 72
Трудоемкость E = 316009
Время T = 17556 сек

Рисунок 2 — Продолжение окна программы

4 РЕЗУЛЬТАТ РАСЧЕТА МЕТРИК

Операторы		Операторы		Операторы	
;	44	if...else	4	Color	3
{ }	27	?	2	RED	2
9	8	:	7	GREEN	3
()	19	while	1	BLUE	2
<=	4	--	1	Point	3
return	14	break	1	std	1
*	4	try	1	x	5
factorial()	2	goto	1	y	5
-	1	/	1	n	7
=	19	[]	1	1	10
square()	4	throw	1	a	13
power()	3	runtime_error()	2	base	3
++	5	catch	1	exp	3
for	5	what()	1	2	3
*=	2	&&	1	result	4
movePoint()	2	==	8	i	14
j	3	if	4	0	9
.	3	<	4	p	3
isPositive()	2	+	3	dx	2
>	2	+=	3	dy	2
colorName()	2	do...while	1	value	5
switch	1	continue	1	nullptr	1
case	3			false	1
default	1			c	2
main()	1			"красный"	1
%	3			"Зелёный"	1
<<	42			"синий"	1
				"Неизвестный"	1

Рисунок 3 – Расчет метрик вручную

15	2	sum	3
count	19	num	2
"FizzBuzz"	1	"Сумма числа:"	1
end	18	"Таблица to break/continue:"	1
3	7	j	7
"Fizz"	1	"x"	1
5	5	"="	1
"Buz"	1	pl	1
10	2	7	1
20	1	"Точка после перемножения:"	1
maxVal	2	" , "	1
"Максимум:"	1	") "	1
myColor	2	1/2	1
"Цвет:"	1	ptr	3
"Квадрат чисел"	1	"Знак через указатель:"	1
" "	2	"Получившееся?"	1
fact	3	"ga"	1
counter	5	"nem"	1
"пример "через while:"	1	"2^5 ="	1
"пример "через рекурсию"	1	"3^2 (по формуле числа)"	1
attempts	4	"square(2.5) ="	1
"Число попыток:"	1	2.5	1
numbers	2	divisor	3
4	1	"Результат на поле:"	1
6	1		

Рисунок 4 – Продолжение расчета метрик вручную

оператор
 100 1
 e 2
 "Ошибки" 1
 start_label 2
 "goto оператор" 1
 83 227

$$\begin{aligned}
 n &= 48 + 83 = 131 \text{ - словарь программы} \\
 N &= 262 + 227 = 489 \text{ - длина программы} \\
 V &= 489 \cdot \log_2(131) \approx 489 \cdot 7.033 = 3439 \text{ - объем программы} \\
 D &= \frac{48}{2} \cdot \frac{227}{83} \approx 24 \cdot 2.735 = 66 \text{ - сложность} \\
 E &= 66 \cdot 3439 = 226974 \text{ - трудоемкость} \\
 T &= \frac{226974}{18} = 12609 \text{ сек - время реализации}
 \end{aligned}$$

из исследования
 Лопатева

Рисунок 5 – Продолжение расчета метрик вручную

Программа-парсер написана на языке Python с использованием стандартной библиотеки tkinter с модулем ttk для создания графической части. Результатом работы программы является вывод таблицы операторов и операндов, а также результата расчета базовых и расширенных метрик Холстеда. Результат работы программы представлен на рисунках 1-2.